

Numerical Methods for Engineering Computation

MATLAB Practice Sheet 1

Root-Finding Methods

Verified Questions, Generic Solvers, and Implementation Practice

Bisection • Fixed Point • Newton • Modified Newton
Secant • False Position • Muller

Contents

1	Purpose and Learning Outcomes	3
2	Concept Map and Method Selection	3
3	Verified MATLAB-1 Coverage Checklist	5
3.1	Questions 1–8: Bisection	5
3.2	Questions 9–15: Fixed-Point Iteration and Application	6
3.3	Questions 16–22: Newton, Secant and False Position	6
3.4	Questions 23–26: Multiple Roots, Muller and Cross-Method Comparison	7
4	Generic MATLAB Code Library	8
4.1	Bisection: <code>bisection_nm.m</code>	8
4.2	Fixed-Point Iteration: <code>fixedpoint_nm.m</code>	8
4.3	Newton: <code>newton_nm.m</code>	9
4.4	Modified Newton for multiple roots: <code>modified_newton_nm.m</code>	9
4.5	Secant: <code>secant_nm.m</code>	9
4.6	False Position: <code>falseposition_nm.m</code>	10
4.7	Muller: <code>muller_nm.m</code>	10
4.8	Generic driver template	11
5	Bisection Method - Question-wise Solutions	12
5.1	Question 1	12
5.2	Question 2(a)	13
5.3	Question 2(b)	13
5.4	Question 2(c)	14
6	Newton Method - Question-wise Solutions	15
6.1	Question 1(a)	15
6.2	Question 1(b)	15
6.3	Question 1(c)	16
6.4	Question 1(d)	17
7	Secant Method - Question-wise Solutions	18
7.1	Question 1(a)	18
7.2	Question 1(b)	18
7.3	Question 1(c)	19
7.4	Question 1(d)	19
8	False-Position Method - Question-wise Solutions	21
8.1	Reusable False-Position Function	21
8.2	Questions 1(a)-1(d)	21

9 Method Comparison	22
10 Practice Problems to Implement	23
10.1 A. Bisection and False Position	23
10.2 B. Fixed-Point Iteration	23
10.3 C. Newton and Modified Newton	23
10.4 D. Secant Method	24
10.5 E. Muller Method	24
10.6 F. Method Selection and Engineering Integration	25
11 Compact Formula Sheet	26
12 Suggested Student Submission	26

1 Purpose and Learning Outcomes

Learning Outcomes

After completing this sheet, a student should be able to:

- LO1.** write scalar nonlinear equations in the form $f(x) = 0$;
- LO2.** implement Bisection, Fixed-Point, Newton, Modified Newton, Secant, False-Position, and Muller algorithms in MATLAB;
- LO3.** use question-wise main/function-file organisation together with reusable generic solver functions;
- LO4.** interpret the approximate root, residual, iteration count, bracket width, and convergence behaviour;
- LO5.** select a bracketing, open, multiple-root, or quadratic-interpolation method appropriately;
- LO6.** verify every reported root by evaluating a residual rather than relying only on iteration counts.

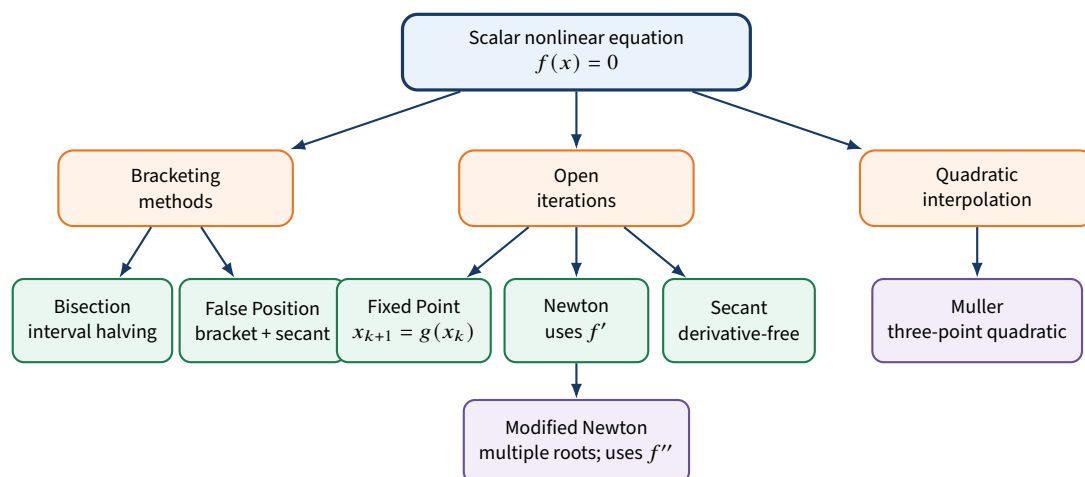
Laboratory Instructions

Each solved item is deliberately separated into three blocks:

1. **Question** - mathematical problem and prescribed interval;
2. **MATLAB files** - a main script and the associated function file; Newton problems also contain a derivative file;
3. **Numerical check** - expected root and a short verification note.

Run the main script only after saving the associated function files in the same working folder.

2 Concept Map and Method Selection



Concept-map interpretation

The seven methods form four practical ideas. **Bisection** and **False Position** preserve a bracket. **Fixed Point**, **Newton**, and **Secant** are open iterations. **Modified Newton** is designed to recover fast convergence near multiple roots. **Muller** replaces a straight-line model by a quadratic model and may naturally enter complex arithmetic.

Situation	Preferred first method	Reason
Reliable sign-change bracket	Bisection	Highest robustness and transparent error bound.
Bracket available but faster endpoint interpolation desired	False Position	Retains bracketing while using a secant-line intercept.
Good initial guess and derivative available	Newton	Usually quadratic near a simple root.
Derivative unavailable	Secant	Approximates the derivative from two iterates.
Natural rearrangement $x = g(x)$ with $g' < 1$	Fixed Point	Simple iteration and clear contraction interpretation.
Known or suspected multiple root	Modified Newton	Avoids the typical linear slowdown of ordinary Newton.
Polynomial/complex-root exploration	Muller	Quadratic interpolation and complex iterates are allowed.

3 Verified MATLAB-1 Coverage Checklist

What was double-checked

The laboratory material and its accompanying exercise sheet were checked together. The laboratory file list explicitly organises question-wise files for Bisection, Newton and Secant, while the accompanying practice set extends the root-finding coverage to Fixed-Point iteration, Modified Newton and Muller. The checklist below preserves the complete mathematical coverage in a compact form and adds direct residual checks where the source typesetting or printed check value is ambiguous.

Important verification corrections

Several source-rendered mathematical expressions contain typographical or OCR ambiguities. For a dependable laboratory sheet, use these verified interpretations:

1. For the Newton test with root near 0.739085, use $f(x) = x - \cos x$.
2. The equation whose root is near 1.512135 is $3x - e^x = 0$.
3. For $2x + 3 \cos x - e^x = 0$, direct substitution shows that the literal equation has root $x \approx 1.239715$, not the printed check 0.976773. Always report $|f(x)|$.
4. In the multiple-root problem, interpret the second function as $\cos(x + \sqrt{2}) + x(x/2 + \sqrt{2})$; its multiple root is $-\sqrt{2}$.
5. For the logarithmic multi-root problem use $\ln(x^2 + 1) - e^{0.4x} \cos(\pi x)$.
6. The exponential plotting problem is $3^{3x+1} - 7 \cdot 5^{2x} = 0$.
7. For the Bisection bound on $x^3 - x - 1 = 0$ over $[1, 2]$, a literal tolerance of 10^{-5} requires at least 17 interval halvings under the standard bound $|p_n - p| \leq (b - a)/2^n$.

These notes are included so that students learn to verify numerical answers rather than copy an iteration number without checking the residual.

3.1 Questions 1–8: Bisection

Q	Task summary	Numerical check / observation
1	Perform three Bisection steps for $\sqrt{x} - \cos x = 0$ on $[0, 1]$.	$p_3 = 0.625$.
2	Solve $x^3 - 7x^2 + 14x - 6 = 0$ to 10^{-2} on $[0, 1]$, $[1, 3.2]$, and $[3.2, 4]$.	Roots $2 - \sqrt{2} \approx 0.585786$, 3, and $2 + \sqrt{2} \approx 3.414214$.
3	Use Bisection to 10^{-5} for: (a) $x - 2^{-x} = 0$ on $[0, 1]$; (b) $e^x - x^2 + 3x - 2 = 0$ on $[0, 1]$; (c) $2x \cos(2x) - (x + 1)^2 = 0$ on $[-3, -2]$ and $[-1, 0]$; (d) $x \cos x - 2x^2 + 3x - 1 = 0$ on $[0.2, 0.3]$ and $[1.2, 1.3]$.	0.641186; 0.257530; -2.191308, -0.798160; 0.297530, 1.256623.
4	Plot $y = x$ and $y = 2 \sin x$, then locate the first positive solution of $x = 2 \sin x$.	$x \approx 1.895494$.
5	Plot $y = e^x - 2$ and $y = \cos(e^x - 2)$ and solve their intersection on $[0.5, 1.5]$.	$x \approx 1.007624$.
6	Examine Bisection behaviour for $f(x) = (x + 2)(x + 1)^2 x(x - 1)^3(x - 2)$ on several wide intervals.	Demonstrates sensitivity to interval/midpoint and even-multiplicity roots; verify the attained root by residual.
7	Approximate $\sqrt[3]{25}$ by solving $x^3 - 25 = 0$ on $[2, 3]$ to 10^{-4} .	2.924018.

Q	Task summary	Numerical check / observation
8	Find an iteration bound and approximate the root of $x^3 - x - 1 = 0$ on $[1, 2]$.	$x \approx 1.324718$; for 10^{-5} use $n \geq 17$ with the standard bound.

3.2 Questions 9–15: Fixed-Point Iteration and Application

Q	Task summary	Numerical check / observation
9	Solve $x^4 - 3x^2 - 3 = 0$ on $[1, 2]$ with $p_0 = 1$, for example with $g(x) = (3x^2 + 3)^{1/4}$.	A source-tolerance iterate is $p_6 \approx 1.943317$; the converged positive root is ≈ 1.947123 .
10	For $g(x) = \pi + \frac{1}{2} \sin(x/2)$ on $[0, 2\pi]$, establish a contraction bound and iterate to 10^{-2} .	Early iterate $p_3 \approx 3.626996$; fixed point ≈ 3.626943 .
11	Approximate $\sqrt{3}$ to 10^{-4} using $g(x) = \frac{1}{2}(x + 3/x)$ with $p_0 = 1$.	$p_4 \approx 1.73205$.
12	For each rearrangement, choose/verify a convergent interval and iterate to 10^{-5} : (a) $x = (2 - e^x + x^2)/3$; (b) $x = 5/x^2 + 2$; (c) $x = \sqrt{e^x/3}$; (d) $x = 5^{-x}$; (e) $x = 6^{-x}$; (f) $x = \frac{1}{2}(\sin x + \cos x)$.	Approximate fixed points: 0.257531, 2.690650, 0.909999, 0.469625, 0.448059, 0.704812.
13	Find all real zeros of $x^2 + 10 \cos x = 0$ using suitable fixed-point rearrangements and different initial values.	$x \approx \pm 1.96882, \pm 3.16193$.
14	Use fixed-point iteration on $2 \sin(\pi x) + x = 0$ in $[1, 2]$ from $p_0 = 1$.	A source stopping iterate is near 1.683855; the converged root is 1.681975 (approximately).
15	Falling-body model: $s(t) = s_0 - \frac{mg}{k}t + \frac{m^2g}{k^2}(1 - e^{-kt/m})$ with $s_0 = 300, m = 0.25, k = 0.1, g = 32.17$. Solve $s(t) = 0$ to 0.01 s.	Fixed-point form gives $t \approx 6.0028$ s.

3.3 Questions 16–22: Newton, Secant and False Position

Q	Task summary	Numerical check / observation
16	Newton to 10^{-4} for: (a) $x^3 - 2x^2 - 5 = 0$ on $[1, 4]$; (b) $x^3 + 3x^2 - 1 = 0$ on $[-3, -2]$; (c) $x - \cos x = 0$ on $[0, \pi/2]$; (d) $x - 0.8 - 0.2 \sin x = 0$ on $[0, \pi/2]$.	2.690647, -2.879385, 0.739085, 0.964334.
17	Repeat Q16 with Secant, taking interval endpoints as the first two approximations.	Same roots; compare iteration counts and residuals.
18	Repeat Q16 with False Position.	Same roots; note possible one-sided endpoint stagnation.
19	Compare Newton, Secant and False Position for (a) $3x - e^x = 0$ near $[1, 2]$ and (b) the printed $2x + 3 \cos x - e^x = 0$.	(a) 1.512135. For literal (b), residual verification gives 1.239715; do not accept an inconsistent check value without substitution.
20	Use the three methods for both real zeros of $230x^4 + 18x^3 + 9x^2 - 221x - 9 = 0$ on $[-1, 0]$ and $[0, 1]$.	-0.04065929 and 0.96239842.
21	For $f(x) = \ln(x^2 + 1) - e^{0.4x} \cos(\pi x)$ find the negative zero, first four positive zeros, devise an initial-guess pattern, and obtain the 25th positive zero.	-0.434143; 0.450657, 1.744738, 2.238320, 3.709041; 25th ≈ 24.499887 .

Q	Task summary	Numerical check / observation
22	Plot and solve $3^{3x+1} - 7 \cdot 5^{2x} = 0$ to very high precision.	$x \approx 11.00943864426817$.

3.4 Questions 23–26: Multiple Roots, Muller and Cross-Method Comparison

Q	Task summary	Numerical check / observation
23	Compare ordinary and Modified Newton for: (a) $(x - e^{-x})^2$; (b) $\cos(x + \sqrt{2}) + x(x/2 + \sqrt{2})$; (c) $(x - 2^{-x})^3$; (d) $e^{6x} + 3(\ln 2)^2 e^{2x} - (\ln 8)e^{4x} - (\ln 2)^3$.	Multiple roots approximately 0.567143, $-\sqrt{2} = -1.414214$, 0.641186, -0.183256 ; Modified Newton should require markedly fewer iterations.
24	Repeat the last exponential problem after replacing coefficients by 1.441, 2.079, and 0.3330.	Root ≈ -0.169607 ; compare modelling/coefficient error with iteration error.
25	Apply Bisection, Newton, Secant, False Position and Muller to $600x^4 - 550x^3 + 200x^2 - 20x - 1 = 0$ on $[0.1, 1]$.	Root in the interval ≈ 0.232352965 ; report residual as well as step size.
26	Use Newton and Muller to locate the real zeros of: (a) $x^3 - 2x^2 - 5$; (b) $x^3 + 3x^2 - 1$; (c) $x^3 - x - 1$; (d) $x^4 + 2x^2 - x - 3$; (e) $x^3 + 4.001x^2 + 4.002x + 1.101$; (f) $x^5 - x^4 + 2x^3 - 3x^2 + x - 4$.	Real roots, respectively: (a) 2.690647; (b) $-2.879385, -0.652704, 0.532089$; (c) 1.324718; (d) $-0.876053, 1.124123$; (e) $-2.645606, -0.885330, -0.470064$; (f) 1.498190.

Minimum evidence for every question

For each computed root x_* , the student report should contain: method, starting data, tolerance, number of iterations, x_* , $|f(x_*)|$, and one convergence diagnostic (bracket width or $|x_{k+1} - x_k|$). This catches transcription errors immediately.

4 Generic MATLAB Code Library

Reusable solver philosophy

The following functions follow the same design used in the textbook annexure: validate assumptions, protect dangerous divisions, cap iteration counts, store iteration history, and return a convergence flag and residual. Save each function in a separate .m file with the same name as the function.

4.1 Bisection: bisection_nm.m

```

1 function [root,info] = bisection_nm(f,a,b,tol,maxit)
2 fa = f(a); fb = f(b);
3 if ~isfinite(fa) || ~isfinite(fb)
4     error('Endpoint function values must be finite.');
```

4.2 Fixed-Point Iteration: fixedpoint_nm.m

```

1 function [x,info] = fixedpoint_nm(g,f,x0,tol,maxit)
2 x = x0; hist = zeros(maxit,5); converged = false;
3 for k = 1:maxit
4     xnew = g(x);
5     if ~isfinite(xnew)
6         error('Fixed-point map produced a non-finite iterate.');
```

Before using Fixed Point

A rearrangement $x = g(x)$ is not automatically a good numerical method. Check that g maps the intended interval into itself and, whenever possible, verify a contraction condition $\max |g'(x)| < 1$ on that interval.

4.3 Newton: newton_nm.m

```

1 function [x,info] = newton_nm(f,df,x0,tol,maxit)
2 x = x0; hist = zeros(maxit,5); converged = false;
3 for k = 1:maxit
4     fx = f(x); dfx = df(x);
5     if ~isfinite(fx) || ~isfinite(dfx)
6         error('Non-finite function or derivative value.');
7     end
8     if abs(dfx) <= sqrt(eps)*(1+abs(fx))
9         error('Derivative too small for a reliable Newton step.');
10    end
11    step = -fx/dfx; xnew = x+step; fxnew = f(xnew);
12    hist(k,:) = [k,x,fx,step,fxnew];
13    if abs(step) <= tol*(1+abs(xnew)) && abs(fxnew) <= tol
14        x = xnew; converged = true; break
15    end
16    x = xnew;
17 end
18 info.iterations = k;
19 info.converged = converged;
20 info.residual = abs(f(x));
21 info.history = array2table(hist(1:k,:), ...
22     'VariableNames',{'k','x','f_x','step','f_xNew'});
23 end

```

4.4 Modified Newton for multiple roots: modified_newton_nm.m

```

1 function [x,info] = modified_newton_nm(f,df,ddf,x0,tol,maxit)
2 x = x0; hist = zeros(maxit,6); converged = false;
3 for k = 1:maxit
4     fx = f(x); dfx = df(x); ddfx = ddf(x);
5     den = dfx^2 - fx*ddfx;
6     if abs(den) <= sqrt(eps)*(1+abs(dfx)^2+abs(fx*ddfx))
7         error('Modified-Newton denominator is too small.');
8     end
9     step = -(fx*dfx)/den; xnew = x+step; fxnew = f(xnew);
10    hist(k,:) = [k,x,fx,dfx,step,fxnew];
11    if abs(step) <= tol*(1+abs(xnew)) && abs(fxnew) <= tol
12        x = xnew; converged = true; break
13    end
14    x = xnew;
15 end
16 info.iterations = k;
17 info.converged = converged;
18 info.residual = abs(f(x));
19 info.history = array2table(hist(1:k,:), ...
20     'VariableNames',{'k','x','f_x','df_x','step','f_xNew'});
21 end

```

4.5 Secant: secant_nm.m

```

1 function [x,info] = secant_nm(f,x0,x1,tol,maxit)
2 f0 = f(x0); f1 = f(x1); hist = zeros(maxit,6); converged = false;
3 for k = 1:maxit
4     den = f1-f0;
5     if abs(den) <= eps*(1+abs(f0)+abs(f1))
6         error('Secant denominator is too small.');
```

4.6 False Position: falseposition_nm.m

```

1 function [root,info] = falseposition_nm(f,a,b,tol,maxit)
2 fa = f(a); fb = f(b);
3 if fa*fb > 0, error('A sign-changing bracket is required.');
```

4.7 Muller: muller_nm.m

```

1 function [x,info] = muller_nm(f,x0,x1,x2,tol,maxit)
2 hist = complex(zeros(maxit,6)); converged = false;
3 for k = 1:maxit
4     f0=f(x0); f1=f(x1); f2=f(x2);
```

```

5  h1=x1-x0; h2=x2-x1;
6  if h1==0 || h2==0 || h1+h2==0
7      error('Muller points must be distinct.');
```

end

```

9  d1=(f1-f0)/h1; d2=(f2-f1)/h2;
10 d=(d2-d1)/(h2+h1);
11 b=d2+h2*d;
12 D=sqrt(complex(b^2-4*f2*d));
13 if abs(b-D) < abs(b+D), E=b+D; else, E=b-D; end
14 if abs(E) <= eps*(1+abs(b)+abs(D))
15     error('Muller denominator is too small.');
```

end

```

17 h=-2*f2/E; x3=x2+h; f3=f(x3);
18 hist(k,:)=[k,x0,x1,x2,x3,h];
19 if abs(h) <= tol*(1+abs(x3)) && abs(f3) <= tol
20     x2=x3; converged=true; break
21 end
22 x0=x1; x1=x2; x2=x3;
23 end
24 x=x2;
25 info.iterations=k;
26 info.converged=converged;
27 info.residual=abs(f(x));
28 info.history=array2table(hist(1:k,:), ...
29     'VariableNames',{'k','x0','x1','x2','x3','step'});
30 end
```

Muller and complex arithmetic

Do not force the discriminant to be real. The use of `sqrt(complex(...))` is intentional: a quadratic interpolant can produce complex iterates even when the original coefficients are real. This is an advantage when polynomial roots are sought.

4.8 Generic driver template

```

1  clear; clc; close all; format long g
2
3  tol = 1e-8; maxit = 100;
4  f = @(x) x.^3 - x - 1;
5  df = @(x) 3*x.^2 - 1;
6
7  % Choose ONE method at a time:
8  [rB, iB] = bisection_nm(f,1,2,tol,maxit);
9  [rN, iN] = newton_nm(f,df,1.5,tol,maxit);
10 [rS, iS] = secant_nm(f,1,2,tol,maxit);
11 [rF, iF] = falseposition_nm(f,1,2,tol,maxit);
12
13 fprintf('Bisection : %.12f residual %.3e it %d\n', ...
14     rB,iB.residual,iB.iterations);
15 fprintf('Newton    : %.12f residual %.3e it %d\n', ...
16     rN,iN.residual,iN.iterations);
17 fprintf('Secant     : %.12f residual %.3e it %d\n', ...
18     rS,iS.residual,iS.iterations);
19 fprintf('False Pos.: %.12f residual %.3e it %d\n', ...
20     rF,iF.residual,iF.iterations);
```

Expected check for the driver

All four methods should converge to the plastic constant

$$x_* \approx 1.324717957244746,$$

with a small residual $|x_*^3 - x_* - 1|$. Iteration counts differ because the methods have different convergence mechanisms.

5 Bisection Method - Question-wise Solutions

Bisection update

For a continuous function with $f(a)f(b) < 0$, set $p = (a + b)/2$ and retain the half-interval that still contains a sign change.

5.1 Question 1

Q1 - Three bisection steps

For $f(x) = \sqrt{x} - \cos x$ on $[0, 1]$, perform three Bisection iterations and report the third midpoint.

main1.m

```

1 clc; clear;
2 a = 0; b = 1;
3 n = 3;
4
5 fprintf('iter      a      b      p      f(p)\n');
6 for i = 1:n
7     p = (a+b)/2;
8     fprintf('%3d  %10.6f %10.6f %10.6f %12.6e\n', ...
9             i,a,b,p,function1(p));
10    if function1(a)*function1(p) < 0
11        b = p;
12    else
13        a = p;
14    end
15 end
16 fprintf('\nThird midpoint = %.6f\n',p);

```

function1.m

```

1 function y = function1(x)
2 y = sqrt(x) - cos(x);
3 end

```

Numerical Check

The successive midpoints are 0.5, 0.75, and 0.625. Hence the third midpoint approximation is

$$p_3 = 0.625000.$$

5.2 Question 2(a)

Q2(a) - Root on [0, 1]

Use Bisection for $x^3 - 7x^2 + 14x - 6 = 0$ on $[0, 1]$ with tolerance 10^{-2} .

main2a.m

```

1 clc; clear;
2 a = 0; b = 1; tol = 1e-2;
3 i = 0;
4 while (b-a)/2 > tol
5     i = i+1;
6     p = (a+b)/2;
7     if function2a(a)*function2a(p) < 0
8         b = p;
9     else
10        a = p;
11    end
12 end
13 p = (a+b)/2;
14 fprintf('Root = %.8f, iterations = %d, residual = %.3e\n', ...
15        p,i,abs(function2a(p)));

```

function2a.m

```

1 function y = function2a(x)
2 y = x.^3 - 7*x.^2 + 14*x - 6;
3 end

```

Numerical Check

The root in this interval is approximately 0.5858. A Bisection approximation meeting the requested interval-width tolerance is close to this value.

5.3 Question 2(b)

Q2(b) - Root on [1, 3.2]

Use the same polynomial on $[1, 3.2]$ with tolerance 10^{-2} .

main2b.m

```

1 clc; clear;
2 a = 1; b = 3.2; tol = 1e-2;
3 i = 0;
4 while (b-a)/2 > tol
5     i = i+1;
6     p = (a+b)/2;
7     if function2b(a)*function2b(p) < 0
8         b = p;
9     else
10        a = p;
11    end
12 end
13 p = (a+b)/2;

```

```

14 fprintf('Root = %.8f, iterations = %d, residual = %.3e\n', ...
15         p,i,abs(function2b(p)));

```

function2b.m

```

1 function y = function2b(x)
2 y = x.^3 - 7*x.^2 + 14*x - 6;
3 end

```

Numerical Check

The interval isolates the exact root $x = 3$. Numerically the algorithm converges to a value close to 3.0000.

5.4 Question 2(c)

Q2(c) - Root on [3.2, 4]

Use the same polynomial on [3.2, 4] with tolerance 10^{-2} .

main2c.m

```

1 clc; clear;
2 a = 3.2; b = 4; tol = 1e-2;
3 i = 0;
4 while (b-a)/2 > tol
5     i = i+1;
6     p = (a+b)/2;
7     if function2c(a)*function2c(p) < 0
8         b = p;
9     else
10        a = p;
11    end
12 end
13 p = (a+b)/2;
14 fprintf('Root = %.8f, iterations = %d, residual = %.3e\n', ...
15         p,i,abs(function2c(p)));

```

function2c.m

```

1 function y = function2c(x)
2 y = x.^3 - 7*x.^2 + 14*x - 6;
3 end

```

Numerical Check

The third real root is approximately

$$x \approx 3.4142.$$

6 Newton Method - Question-wise Solutions

Newton update

$$p_{n+1} = p_n - \frac{f(p_n)}{f'(p_n)}.$$

The derivative must not be zero or numerically too small at the current iterate.

6.1 Question 1(a)

Q1(a)

Find the root of $x^3 - 2x^2 - 5 = 0$ in $[1, 4]$ to accuracy 10^{-4} using Newton's method.

newton_main1a.m

```
1 clc; clear;
2 p = 2.5; tol = 1e-4; maxit = 100;
3 for i = 1:maxit
4     pnew = p - newton_function1a(p)/newton_derivativel1a(p);
5     fprintf('%3d  %.10f  %.3e\n', i, pnew, abs(newton_function1a(pnew)));
6     if abs(pnew-p) < tol, break; end
7     p = pnew;
8 end
9 fprintf('Root = %.10f\n', pnew);
```

newton_function1a.m

```
1 function y = newton_function1a(x)
2 y = x.^3 - 2*x.^2 - 5;
3 end
```

newton_derivative1a.m

```
1 function y = newton_derivativel1a(x)
2 y = 3*x.^2 - 4*x;
3 end
```

Numerical Check

$$x \approx 2.6906474480$$

With the midpoint $p_0 = 2.5$, the tolerance is reached rapidly.

6.2 Question 1(b)

Q1(b)

Find the root of $x^3 + 3x^2 - 1 = 0$ in $[-3, -2]$ to accuracy 10^{-4} using Newton's method.

newton_main1b.m

```

1 clc; clear;
2 p = -2.5; tol = 1e-4; maxit = 100;
3 for i = 1:maxit
4     pnew = p - newton_function1b(p)/newton_derivative1b(p);
5     fprintf('%3d  %.10f  %.3e\n',i,pnew,abs(newton_function1b(pnew)));
6     if abs(pnew-p) < tol, break; end
7     p = pnew;
8 end
9 fprintf('Root = %.10f\n',pnew);

```

newton_function1b.m

```

1 function y = newton_function1b(x)
2 y = x.^3 + 3*x.^2 - 1;
3 end

```

newton_derivative1b.m

```

1 function y = newton_derivative1b(x)
2 y = 3*x.^2 + 6*x;
3 end

```

Numerical Check

$$x \approx -2.8793852416.$$

6.3 Question 1(c)**Q1(c)**

Solve $x - \cos x = 0$ on $[0, \pi/2]$ to accuracy 10^{-4} using Newton's method.

newton_main1c.m

```

1 clc; clear;
2 p = pi/4; tol = 1e-4; maxit = 100;
3 for i = 1:maxit
4     pnew = p - newton_function1c(p)/newton_derivative1c(p);
5     fprintf('%3d  %.10f  %.3e\n',i,pnew,abs(newton_function1c(pnew)));
6     if abs(pnew-p) < tol, break; end
7     p = pnew;
8 end
9 fprintf('Root = %.10f\n',pnew);

```

newton_function1c.m

```

1 function y = newton_function1c(x)
2 y = x - cos(x);
3 end

```

newton_derivative1c.m

```

1 function y = newton_derivative1c(x)
2 y = 1 + sin(x);
3 end

```

Numerical Check

$$x \approx 0.7390851332.$$

6.4 Question 1(d)**Q1(d)**

Solve $x - 0.8 - 0.2 \sin x = 0$ on $[0, \pi/2]$ to accuracy 10^{-4} using Newton's method.

newton_main1d.m

```

1 clc; clear;
2 p = pi/4; tol = 1e-4; maxit = 100;
3 for i = 1:maxit
4     pnew = p - newton_function1d(p)/newton_derivative1d(p);
5     fprintf('%3d %.10f %.3e\n', i, pnew, abs(newton_function1d(pnew)));
6     if abs(pnew-p) < tol, break; end
7     p = pnew;
8 end
9 fprintf('Root = %.10f\n', pnew);

```

newton_function1d.m

```

1 function y = newton_function1d(x)
2 y = x - 0.8 - 0.2*sin(x);
3 end

```

newton_derivative1d.m

```

1 function y = newton_derivative1d(x)
2 y = 1 - 0.2*cos(x);
3 end

```

Numerical Check

$$x \approx 0.9643338877.$$

7 Secant Method - Question-wise Solutions

Secant update

$$p_n = p_{n-1} - f(p_{n-1}) \frac{p_{n-1} - p_{n-2}}{f(p_{n-1}) - f(p_{n-2})}.$$

The endpoints of the indicated interval are used as the two initial approximations.

7.1 Question 1(a)

Q1(a)

Apply the Secant method to $x^3 - 2x^2 - 5 = 0$ using $p_0 = 1$ and $p_1 = 4$.

secant_main1a.m

```
1 clc; clear;
2 p0 = 1; p1 = 4; tol = 1e-4; maxit = 100;
3 for i = 1:maxit
4     f0 = secant_function1a(p0); f1 = secant_function1a(p1);
5     p = p1 - f1*(p1-p0)/(f1-f0);
6     fprintf('%3d %.10f %.3e\n',i,p,abs(secant_function1a(p)));
7     if abs(p-p1) < tol, break; end
8     p0 = p1; p1 = p;
9 end
10 fprintf('Root = %.10f\n',p);
```

secant_function1a.m

```
1 function y = secant_function1a(x)
2 y = x.^3 - 2*x.^2 - 5;
3 end
```

Numerical Check

$$x \approx 2.690647448$$

7.2 Question 1(b)

Q1(b)

Apply the Secant method to $x^3 + 3x^2 - 1 = 0$ using $p_0 = -3$ and $p_1 = -2$.

secant_main1b.m

```
1 clc; clear;
2 p0 = -3; p1 = -2; tol = 1e-4; maxit = 100;
3 for i = 1:maxit
4     f0 = secant_function1b(p0); f1 = secant_function1b(p1);
5     p = p1 - f1*(p1-p0)/(f1-f0);
6     fprintf('%3d %.10f %.3e\n',i,p,abs(secant_function1b(p)));
7     if abs(p-p1) < tol, break; end
```

```

8     p0 = p1; p1 = p;
9 end
10 fprintf('Root = %.10f\n',p);

```

secant_function1b.m

```

1 function y = secant_function1b(x)
2 y = x.^3 + 3*x.^2 - 1;
3 end

```

Numerical Check

$$x \approx -2.879385242$$

7.3 Question 1(c)

Q1(c)

Apply the Secant method to $x - \cos x = 0$ using $p_0 = 0$ and $p_1 = \pi/2$.

secant_main1c.m

```

1 clc; clear;
2 p0 = 0; p1 = pi/2; tol = 1e-4; maxit = 100;
3 for i = 1:maxit
4     f0 = secant_function1c(p0); f1 = secant_function1c(p1);
5     p = p1 - f1*(p1-p0)/(f1-f0);
6     fprintf('%3d %.10f %.3e\n',i,p,abs(secant_function1c(p)));
7     if abs(p-p1) < tol, break; end
8     p0 = p1; p1 = p;
9 end
10 fprintf('Root = %.10f\n',p);

```

secant_function1c.m

```

1 function y = secant_function1c(x)
2 y = x - cos(x);
3 end

```

Numerical Check

$$x \approx 0.739085133$$

7.4 Question 1(d)

Q1(d)

Apply the Secant method to $x - 0.8 - 0.2 \sin x = 0$ using $p_0 = 0$ and $p_1 = \pi/2$.

secant_main1d.m

```
1 clc; clear;
2 p0 = 0; p1 = pi/2; tol = 1e-4; maxit = 100;
3 for i = 1:maxit
4     f0 = secant_function1d(p0); f1 = secant_function1d(p1);
5     p = p1 - f1*(p1-p0)/(f1-f0);
6     fprintf('%3d %.10f %.3e\n',i,p,abs(secant_function1d(p)));
7     if abs(p-p1) < tol, break; end
8     p0 = p1; p1 = p;
9 end
10 fprintf('Root = %.10f\n',p);
```

secant_function1d.m

```
1 function y = secant_function1d(x)
2 y = x - 0.8 - 0.2*sin(x);
3 end
```

Numerical Check

$$x \approx 0.964333888$$

8 False-Position Method - Question-wise Solutions

False-Position update

At every iteration use the secant-line intercept

$$p = b - f(b) \frac{b - a}{f(b) - f(a)},$$

and then retain the subinterval over which the function changes sign.

8.1 Reusable False-Position Function

falsePosition.m

```

1 function [p,k] = falsePosition(fun,a,b,tol,maxit)
2 fa = fun(a); fb = fun(b);
3 if fa*fb > 0
4     error('Initial interval must contain a sign change.');
```

8.2 Questions 1(a)-1(d)

Q1(a)-Q1(d)

Apply False Position to the same four equations used in the Newton and Secant sections, using the given interval endpoints and tolerance 10^{-4} .

falseposition_main1a.m

```

1 clc; clear;
2 f = @(x) x.^3 - 2*x.^2 - 5;
3 [p,k] = falsePosition(f,1,4,1e-4,200);
4 fprintf('Root = %.10f, iterations = %d\n',p,k);
```

falseposition_main1b.m

```

1 clc; clear;
2 f = @(x) x.^3 + 3*x.^2 - 1;
3 [p,k] = falsePosition(f,-3,-2,1e-4,200);
```

```
4 fprintf('Root = %.10f, iterations = %d\n',p,k);
```

falseposition_main1c.m

```
1 clc; clear;
2 f = @(x) x - cos(x);
3 [p,k] = falsePosition(f,0,pi/2,1e-4,200);
4 fprintf('Root = %.10f, iterations = %d\n',p,k);
```

falseposition_main1d.m

```
1 clc; clear;
2 f = @(x) x - 0.8 - 0.2*sin(x);
3 [p,k] = falsePosition(f,0,pi/2,1e-4,200);
4 fprintf('Root = %.10f, iterations = %d\n',p,k);
```

Expected numerical roots

Part	Approximate root
(a)	2.69064745
(b)	-2.87938524
(c)	0.73908513
(d)	0.96433389

9 Method Comparison

Method	Starting information	Major advantage	Main caution
Bisection	Sign-changing interval	Guaranteed interval contraction	Slow linear convergence
Newton	One initial value and derivative	Very fast near a simple root	Sensitive to poor initial values or small derivative
Secant	Two initial values	No derivative required	Can leave a useful region
False Position	Sign-changing interval	Retains a valid bracket	One endpoint may remain nearly fixed

Common Pitfall

Never judge convergence only from the change in successive iterates. Report both the step size and the residual $|f(p)|$. A small step can occur even when the current point is not a satisfactory root.

10 Practice Problems to Implement

Required output for every implementation

For every problem below, submit: (i) MATLAB function handle(s), (ii) chosen method and starting data, (iii) root, (iv) residual, (v) iteration count, (vi) convergence-history table or plot, and (vii) a one-sentence interpretation of why the method succeeded or struggled.

10.1 A. Bisection and False Position

P1 - Drag coefficient

Solve

$$F(c) = \frac{gm}{c} \left(1 - e^{-ct/m}\right) - v = 0$$

for c with $m = 68.1$ kg, $g = 9.81$ m/s², $t = 10$ s, and $v = 40$ m/s. First plot $F(c)$, find a sign-changing bracket, then solve by Bisection and False Position.

P2 - Transcendental benchmark

Solve $xe^x - 3 = 0$ on $[0, 2]$ to 10^{-8} . Compare Bisection and False Position using residual versus iteration count.

P3 - Multiple brackets

Plot $f(x) = \sin(3x) - 0.2x$ on $[-4, 4]$. Automatically detect all sign changes using a coarse grid, then use Bisection on each bracket. Explain which roots could be missed by sign-change detection and why.

10.2 B. Fixed-Point Iteration

P4 - Competing rearrangements

For $x^3 + x - 1 = 0$, design at least three algebraically equivalent maps $x = g(x)$. Estimate $\max |g'(x)|$ on a reasonable interval and test which maps converge from $x_0 = 0.5$.

P5 - Thermal radiation balance

Solve $T^4 + 0.15T - 1 = 0$ for the positive root by constructing a fixed-point map. Justify the interval and contraction numerically before iteration.

P6 - Cobweb study

For $g(x) = \cos x$, start with $x_0 = 0$ and create a cobweb plot together with a semilog plot of $|x_k - x_{k-1}|$. Estimate the observed linear convergence factor.

10.3 C. Newton and Modified Newton

P7 - Simple root

Apply Newton to $f(x) = x^3 - 4x - 9$ from two different starting values. Plot the tangent-line geometry for the first three iterations and compare convergence histories.

P8 - Double root

For $f(x) = (x - 1)^2(x + 2)$, compare ordinary Newton with Modified Newton from $x_0 = 1.8$. Estimate the observed order of convergence from the error history.

P9 - Triple root

Repeat P8 for $f(x) = (x - 0.5)^3(x + 1)$ from $x_0 = 0.9$. Explain why a small residual alone can be misleading near a multiple root.

P10 - Nonlinear spring equilibrium

A spring force is $F_s = kx + \alpha x^3$ with $k = 800$ N/m and $\alpha = 2.0 \times 10^5$ N/m³. Find the positive displacement satisfying $F_s = 1200$ N using Newton. Report displacement in mm.

10.4 D. Secant Method**P11 - Derivative-free comparison**

Solve $e^{-x} - x = 0$ using Secant from $(x_0, x_1) = (0, 1)$ and Newton from $x_0 = 0.5$. Compare function evaluations, not only iterations.

P12 - Poor initial pair

Use the Secant method for $f(x) = \tan x - x$ near the nonzero root between 4 and 5. Try two different starting pairs and discuss the effect of near-vertical tangents.

P13 - Near-equal function values

Construct a case where $f(x_1) \approx f(x_0)$ in a Secant iteration. Demonstrate the denominator safeguard in `secant_nm.m` and explain why it is necessary.

10.5 E. Muller Method**P14 - Real polynomial roots**

Use Muller to find every real root of $x^4 - 3x^3 - x + 3 = 0$. Use different triples of starting points and verify each root with `roots` only after completing your own iteration.

P15 - Complex roots

Apply Muller to $x^4 + 1 = 0$. Demonstrate at least one run that enters the complex plane and compare the computed roots with `roots([1 0 0 0 1])`.

P16 - Closely spaced roots

For $f(x) = (x - 1)(x - 1.001)(x + 2)$, investigate how different initial triples affect which root Muller finds. Report residuals to at least 10^{-10} .

10.6 F. Method Selection and Engineering Integration

P17 - Pipe-flow friction factor

For a simplified Colebrook-type equation

$$\frac{1}{\sqrt{f}} + 2 \log_{10} \left(\frac{\varepsilon/D}{3.7} + \frac{2.51}{Re\sqrt{f}} \right) = 0,$$

use $Re = 10^5$ and $\varepsilon/D = 0.002$. Solve for f by two different methods and justify your preferred one.

P18 - Projectile angle

Ignoring drag, solve $R = v_0^2 \sin(2\theta)/g$ for the smaller launch angle when $R = 60$ m and $v_0 = 30$ m/s. Use radians internally. Compare Bisection and Newton.

P19 - Diode operating point

Solve the nonlinear circuit equation

$$\frac{V_s - V_D}{R} = I_s \left(e^{V_D/(nV_T)} - 1 \right)$$

with $V_s = 5$ V, $R = 1$ k Ω , $I_s = 10^{-12}$ A, $n = 1.8$, $V_T = 0.02585$ V. Plot both sides before root finding.

P20 - Automatic solver study

Create a MATLAB script that receives f , optional derivatives, and optional bracket data, then chooses between Bisection, Newton and Secant using the concept map. Test it on at least five equations from this sheet and produce a comparison table of root, residual, iterations and method selected.

Implementation challenge

For P20, do not call `fzero` as the primary solver. It may be used only as an independent verification after your own method returns a result.

11 Compact Formula Sheet

Method	Update	Core requirement / typical behaviour
Bisection	$x_k = (a_k + b_k)/2$	$f(a)f(b) < 0$; interval error halves each step.
Fixed Point	$x_{k+1} = g(x_k)$	Prefer $g([a, b]) \subset [a, b]$ and $\max g' < 1$.
Newton	$x_{k+1} = x_k - f(x_k)/f'(x_k)$	Quadratic near a simple root under standard smoothness assumptions.
Modified Newton	$x_{k+1} = x_k - \frac{f f'}{(f')^2 - f f''}$	Particularly useful for unknown multiple-root multiplicity.
Secant	$x_{k+1} = x_k - f_k \frac{x_k - x_{k-1}}{f_k - f_{k-1}}$	Derivative-free; superlinear near a simple root.
False Position	$x = b - f(b) \frac{b - a}{f(b) - f(a)}$	Secant intercept with bracketing retained.
Muller	quadratic interpolant through three points	Can converge to real or complex roots.

Recommended stopping evidence

For a computed x_k , use more than one criterion whenever practical:

$$|f(x_k)| \leq \text{TOL}, \quad |x_k - x_{k-1}| \leq \text{TOL}(1 + |x_k|),$$

and, for a bracketing method,

$$\frac{|b_k - a_k|}{2} \leq \text{TOL}.$$

A residual is essential when checking a transcribed answer.

12 Suggested Student Submission

Laboratory record format

For each assigned item include:

1. equation and interval/initial values;
2. MATLAB file names and code;
3. iteration output or convergence plot;
4. final root and residual;
5. one short observation on convergence or robustness.